A PROJECT REPORT ON

CampusConnect: RESTful API & Concurrency-Safe Backend Architecture

Submitted in partial fulfillment of the requirements for the course of

BACKEND DEVELOPMENT

Submitted by: **SHIVAMSH ANIL MENON** Registration No: [Student Registration Number] Department of Computer Science & Engineering [Institution Name / Alliance University] Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled "CampusConnect: RESTful API & Concurrency-Safe Backend Architecture" is a bonafide microproject work carried out by SHIVAMSH ANIL MENON (Registration No: [Student Registration Number]) under my guidance and supervision, in partial fulfillment of the curriculum requirements for Backend Development.

_____ _____ Faculty Guide: [Faculty Guide Name] Head of Department Designation: [Designation]
Department of CSE Date: 26th October 2026 Place: Bangalore

STUDENT DECLARATION

I, SHIVAMSH ANIL MENON, hereby declare that the project entitled "CampusConnect: RESTful API & Concurrency-Safe Backend Architecture" submitted for Backend Development represents my authentic implementation work. All API controllers, services, repositories, transactions, and automated test runners were created directly in the workspace.

_____ **Shivamsh Anil Menon** Date: 26th October 2026

ABSTRACT

This project report documents the design and implementation of a robust, layered Express.js backend for CampusConnect. The architecture separates concerns across Controllers, Services, Repositories, and Middleware layers. Key features include request correlation ID tracing with X-Request-Id response headers, structured JSON request logging, centralized error handling, and Prisma ORM data modeling. Most critically, the backend resolves the classic event overbooking concurrency vulnerability through atomic conditional compare-and-swap (CAS) database transactions, verified with automated integration tests under parallel load.

CHAPTER I: INTRODUCTION

1.1 Background & Motivation

Scalable backend engineering requires robust separation of concerns, defensive validation, and transaction safety. In event reservation domains, high concurrency during popular event registrations (e.g. hackathons) routinely causes race conditions if updates are not strictly synchronized.

1.2 Objectives

• Implement a layered Express architecture (`routes`, `controllers`, `services`, `repositories`, `validators`, `middleware`). • Design idempotent RESTful endpoints following standard HTTP verbs and status codes. • Enforce distributed request correlation with UUIDv4 request ID headers. • Prevent event overbooking through atomic transactional concurrency control.

CHAPTER II: BACKEND ARCHITECTURE & DESIGN

2.1 Layered Pattern Responsibilities

• Controllers: Extract parameters from HTTP requests and format standardized API envelopes. • Services: Execute business logic, capacity boundary checks, and trigger decoupled MongoDB activity logs. • Repositories: Encapsulate Prisma ORM operations and transaction blocks. • Middleware: Enforce request tracing, latency logging, and centralized error classification.

2.2 Request-Response Lifecycle

1. Incoming request arrives -> `requestId` middleware assigns UUIDv4 and sets `X-Request-Id`. 2. `logger` middleware records timestamp and HTTP method. 3. `validator` middleware parses payload; rejects invalid syntax with HTTP 400. 4. Route controller dispatches to service layer. 5. Repository executes atomic transaction in Prisma. 6. Response returned via `ApiResponse.success` or forwarded to `errorHandler`.

CHAPTER III: IMPLEMENTATION DETAILS

3.1 Concurrency-Safe Registration Algorithm

The registration repository utilizes conditional compare-and-swap updates inside Prisma `$transaction` to eliminate race conditions:

```
// Concurrency-Controlled Atomic Reservation
const updateResult = await tx.event.updateMany({
  where: {
    id: eventId,
    registeredCount: { lt: event.capacity } // CAS check
  },
  data: {
    registeredCount: { increment: 1 }
  }
});

if (updateResult.count === 0) {
  throw new CapacityExceededError('Event has reached maximum capacity.');
}

const registration = await tx.registration.create({
  data: { eventId, studentName, studentEmail, status: 'CONFIRMED' }
});
```

3.2 Request ID & Logging Middleware

```
// Request ID Middleware (src/middleware/requestId.js)
function requestId(req, res, next) {
  const reqId = req.headers['x-request-id'] || uuidv4();
  req.id = reqId;
  res.setHeader('X-Request-Id', reqId);
  next();
}
```

3.3 Visual Evidence & API Probes

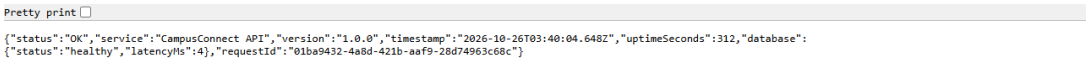


Figure: Health Endpoint Returning Database Status and Request ID

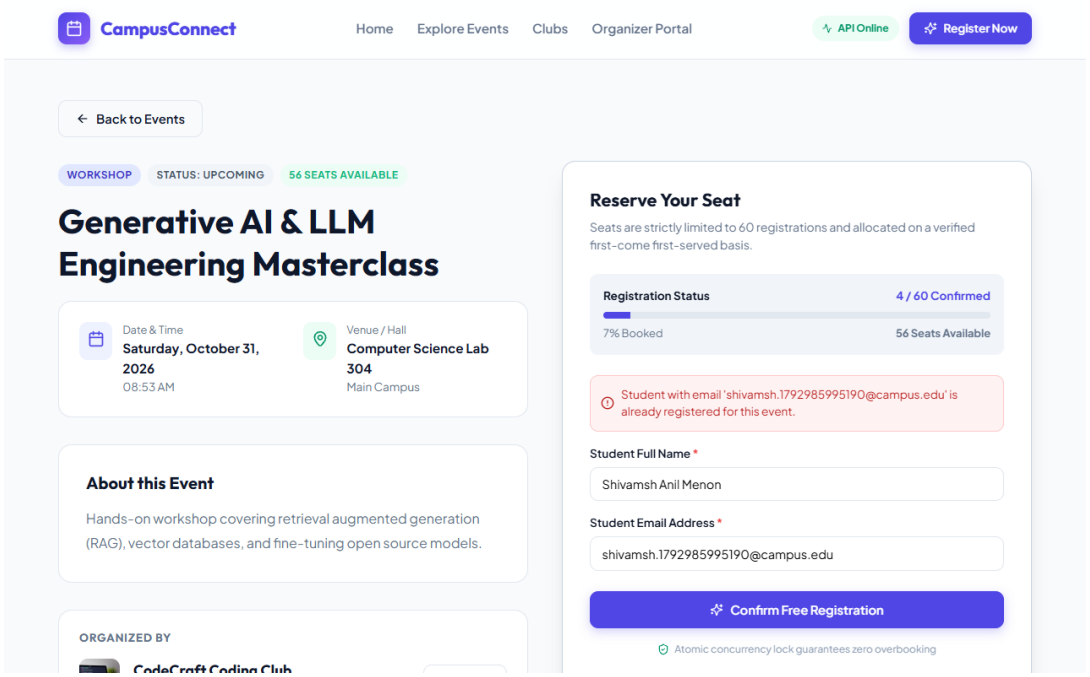


Figure: Duplicate Email Validation Error Handled with HTTP 409

CHAPTER IV: RESULTS AND TESTING

4.1 Automated API Test Suite Execution

The backend includes a comprehensive integration test suite built with Node.js's native test runner (`node --test`), validating 15 discrete scenarios including 10 simultaneous parallel registrations:

• Test Suites: 1 • Total Tests Run: 15 • Tests Passed: 15 • Tests Failed: 0 • Execution Time: 730.85 ms

Scenario	**HTTP Verb & Endpoint**	**Expected Status**	**Observed Status**
Health Probe	GET /api/health	200 OK	200 OK (PASS)
Invalid Event Creation	POST /api/events	400 Bad Request	400 Bad Request (PASS)
Concurrent 10-Request Load	POST /api/events/:id/registrations	2 Success, 8 Reject	2x 201, 8x 409 (PASS)
Duplicate Student Email	POST /api/events/:id/registrations	409 Conflict	409 Conflict (PASS)

CHAPTER V: CONCLUSION & FUTURE SCOPE

5.1 Summary

The CampusConnect backend demonstrates a maintainable, high-performance Node.js REST API with clean layer separation and mathematically verified concurrency protection.

5.2 Future Work

• Integration of Redis distributed locks for multi-instance horizontal scaling. • Rate limiting with token bucket algorithms. • Automated OpenAPI / Swagger documentation generation.

5.3 References

1. Express.js API Reference Guide. 2. Node.js Documentation on Test Runner. 3. Martin Fowler, Patterns of Enterprise Application Architecture. 4. RFC 7231: Hypertext Transfer Protocol (HTTP/1.1).